Cont. Data Structures-I

🎯 Session Objectives

- 📚 Understand the meaning of data structures and why we use them.
- 🗂️ Common data structures in Python
- 📋 Understand what lists are.
- ⚙️ Understand common methods and operations associated with lists.
- 🚀 Understand the meaning of list comprehension
- 🔒 Understand what tuples are.
- 🛠️ Understand common methods and operations associated with tuples.
- ⚖️ Understand the Comparison between Lists and Tuples

```
# 'Joining' strings from a list
word_list = ['Python','is','a','awesome','programming','language']
final_statement = ' '.join(word_list)
print(final_statement)
```

```
Python is a awesome programming language
```

```
char_list = ['P','y','t','h','o','n',' ','P','r','o','g','r','a','m','m','i','n','g','!']
result = ''.join(char_list)
print(result)
```

```
Python Programming!
```

```
# .count() -> Calculating the occurrence of elements in a list....
country_list = ['India','America','Ukraine','New Zealand','Australia','South Africa',
                'Russia','Italy','Spain','Germany','Nepal','Ireland','Iran','Pakistan',
                'Canada','India','America','Germany','China']
count_america = country_list.count('America') # 2
print(count_america)
```

```
2
```

```
count_india = country_list.count('India') # 2
print(count_india)
```

```
2
```

```python
num_list = [1,2,1,2,1,21,2,1,21,12,11,121,22,12,21,22,1211,2121,121]
print(num_list.count(2))
print(num_list.count(1))
```

```
3
4
```

```python
# Deep Copy [Same Memory Address] Vs Shallow Copy [Independent Memory Address]
# .copy() [Shallow Copy]
country_list = ['India','America','Ukraine','New Zealand','Australia','South Africa',
                'Russia','Italy','Spain','Germany','Nepal','Ireland','Iran','Pakistan',
                'Canada','India','America','Germany','China']
new_country_list = country_list.copy() # Shallow Copy
# id() f(x) will return the memory Address of each variable you passed.
print(id(country_list))
print(id(new_country_list))
```

```
2352964268032
2352964239744
```

```python
del new_country_list[-7:]
print("Country List: ", end = " ")
print(country_list)
print("New Country List: ", end = " ")
print(new_country_list) # end = "\n"
```

```
Country List:  ['India', 'America', 'Ukraine', 'New Zealand', 'Australia', 'South Africa', 'Russia', 'Italy',
'Spain', 'Germany', 'Nepal', 'Ireland', 'Iran', 'Pakistan', 'Canada', 'India', 'America', 'Germany', 'China']
New Country List:  ['India', 'America', 'Ukraine', 'New Zealand', 'Australia', 'South Africa', 'Russia', 'Ital
y', 'Spain', 'Germany', 'Nepal', 'Ireland']
```

```python
# deep Copy
weekday_list = ['Mon','Tues','Wed','Thurs','Fri']
weekend_list = ['Sat','Sun']
week_list = weekday_list + weekend_list
new_week_list = week_list # deep copy : as both pointing to the same address
print(id(week_list))
print(id(new_week_list))
```

```
2352924366848
2352924366848
```

```python
# Before
print(week_list)
print(new_week_list)
```

```
['Mon', 'Tues', 'Wed', 'Thurs', 'Fri', 'Sat', 'Sun']
['Mon', 'Tues', 'Wed', 'Thurs', 'Fri', 'Sat', 'Sun']
```

```python
# After
del new_week_list[-3:]
print(week_list)
print(new_week_list)
```

```
['Mon', 'Tues', 'Wed', 'Thurs']
['Mon', 'Tues', 'Wed', 'Thurs']
```

```python
# Using Slicing[:] (Shallow Copy)
country_list = ['India','America','Ukraine','New Zealand','Australia','South Africa',
                'Russia','Italy','Spain','Germany','Nepal','Ireland','Iran','Pakistan',
                'Canada','India','America','Germany','China']

new_country_list = country_list[:] # Shallow Copy
print(id(country_list))
print(id(new_country_list))
```

```
2352964212032
2352964207232
```

```python
# Using list() constructor (Shallow Copy)
country_list = ['India','America','Ukraine','New Zealand','Australia','South Africa',
                'Russia','Italy','Spain','Germany','Nepal','Ireland','Iran','Pakistan',
                'Canada','India','America','Germany','China']

new_country_list = list(country_list) # Shallow Copy
print(id(country_list))
print(id(new_country_list))
```

```
2352964212800
2352964253824
```

```python
print(new_country_list)
```

```
['India', 'America', 'Ukraine', 'New Zealand', 'Australia', 'South Africa', 'Russia', 'Italy', 'Spain', 'Germa
ny', 'Nepal', 'Ireland', 'Iran', 'Pakistan', 'Canada', 'India', 'America', 'Germany', 'China']
```

```python
# Index() -> Its a method to find the position of a first occurrence of a specified value in a list...
country_list = ['India','America','Ukraine','New Zealand','Australia','South Africa',
                'Russia','Italy','Spain','Germany','Nepal','Ireland','Iran','Pakistan',
                'Canada','India','America','Germany','China']

print(country_list.index('India')) # 0
# print(country_list.index('Japan')) # ValueError: 'Japan' is not in list
print(country_list.index('India',1)) # 2nd Occurrence index
print(country_list.index('Germany')) #
print(country_list.index('America',2)) # 2nd Occurrence index
```

```
0
15
9
16
```

```python
print(range(1,11)) # [Start = 1 , Stop = 11-1 = 10 , Step = +1]
```

```
range(1, 11)
```

```python
# List Comprehension ->
squared = [i**2 for i in range(1,11)] # [1,2,3,4.....10]
print(squared)
```

```
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

```python
# List Comprehension ->
squared = [i**2 for i in range(1,11)] # [1,2,3,4.....10]
print(squared)
```

```
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

Memory Address

range(1,11) = [1,2,3...10]

i

Input

[1,2,3,4,..10]

f(X^2)

Output

[1,4,9,16,25,36,49,...100]

i

```python
cube = []
for i in range(1,11): # [1,2,3,4,5,6,7,8,9,10]
    ans = i ** 3
    cube.append(ans)

cube
```

```
[1, 8, 27, 64, 125, 216, 343, 512, 729, 1000]
```

Memory

cube = [1,8,27,64,125,216,343,512,729,1000]

i = ~~1~~ ~~2~~ ~~3~~ ~~4~~ ~~5~~ 10

ans = ~~1~~ ~~8~~ ~~27~~ ~~64~~ ~~125~~ 1000

```python
range(1,11,2) # [1,3,5,7,9]

range(2,11,2) # [2,4,6,8,10]
```

```python
add_10 = [i+10 for i in range(1,11)] # [1,2,3,4.....10]
print(add_10)
```

```
[11, 12, 13, 14, 15, 16, 17, 18, 19, 20]
```

```python
cube = []
for i in range(1,11): # [1,2,3,4,5,6,7,8,9,10]
    ans = i ** 3
    cube.append(ans)

cube
```

```
[1, 8, 27, 64, 125, 216, 343, 512, 729, 1000]
```

```python
cube = [i**3 for i in range(1,11)] # [1,2,3,4.....10]
print(cube)
```

```
[1, 8, 27, 64, 125, 216, 343, 512, 729, 1000]
```

What exactly is a Tuple? ¶

- Ordered Collection (items maintains its positions)
- It can store different type of data types togethers
- Only Difference is: Tuple is an Immutable Container.
- Once Created, their elements can't be changed, added to , or removed.

```
_tuple = ()
print(type(_tuple))
```

```
<class 'tuple'>
```

```
_tuple = (1,2,'3','4','Coding","Python",'k',29.99)
print(_tuple)
```

```
(1, 2, '3', '4', 'Coding', 'Python', 'k', 29.99)
```

```
_tuple = 1,2,3,4,5
print(_tuple)
print(type(_tuple))
```

```
(1, 2, 3, 4, 5)
<class 'tuple'>
```

```
# TypeCasting -> tuple() constructor class
country_list = ['India','America','Ukraine','New Zealand','Australia','South Africa',
                'Russia','Italy','Spain','Germany','Nepal','Ireland','Iran','Pakistan',
                'Canada','India','America','Germany','China']
country_tuple = tuple(country_list)
print(country_tuple)
print(type(country_tuple))
```

```
('India', 'America', 'Ukraine', 'New Zealand', 'Australia', 'South Africa', 'Russia', 'Italy', 'Spain', 'Germa
ny', 'Nepal', 'Ireland', 'Iran', 'Pakistan', 'Canada', 'India', 'America', 'Germany', 'China')
<class 'tuple'>
```

```
country_tuple[-7:]
```

```
('Iran', 'Pakistan', 'Canada', 'India', 'America', 'Germany', 'China')
```

```
country_tuple[-7:] = ('China','Singapore') # Immutable Container
# TypeError: 'tuple' object does not support item assignment
```

```
char_tuple = tuple("Python")
print(char_tuple)
```

```
('P', 'y', 't', 'h', 'o', 'n')
```

```python
countrylist= (['q','r','t'])
print(countrylist)
print(type(countrylist))
```

```
['q', 'r', 't']
<class 'list'>
```

```python
countrylist= (['q','r','t'],)
print(countrylist)
print(type(countrylist)) # 'tuple'
```

```
(['q', 'r', 't'],)
<class 'tuple'>
```

```python
_tuple = tuple(("Python",))
print(_tuple)
```

```
('Python',)
```

```python
nested_list = [
    [11,22,33,44,55],
    ['a','b','c','d','e'],
    [True,True,False,False,True]
]
nested_tuple = tuple(nested_list)
print(nested_tuple)
```

```
([11, 22, 33, 44, 55], ['a', 'b', 'c', 'd', 'e'], [True, True, False, False, True])
```

```python
print(type(nested_tuple))
```

```
<class 'tuple'>
```

```python
nested_tuple[0]
```

```
[11, 22, 33, 44, 55]
```

```python
type(nested_tuple[0]) # 'list'
```

```
list
```

```python
nested_tuple[0].append(66)
print(nested_tuple)
```

```
([11, 22, 33, 44, 55, 66], ['a', 'b', 'c', 'd', 'e'], [True, True, False, False, True])
```

```python
nested_tuple = (('Mon','Tues','Wed'),('Thurs','Fri','Sat','Sun'))
print(nested_tuple)
print(type(nested_tuple))
```

```
(('Mon', 'Tues', 'Wed'), ('Thurs', 'Fri', 'Sat', 'Sun'))
<class 'tuple'>
```

```python
nested_tuple[0]
```

```
('Mon', 'Tues', 'Wed')
```

```python
nested_tuple[0].append('Thurs') # AttributeError
# AttributeError: 'tuple' object has no attribute 'append'
```

```python
nested_tuple[0] = ('Mon','Tues')
# TypeError: 'tuple' object does not support item assignment
```

```python
# Indexing & Slicing # As Tuple is an ordered collection
country_tuple = ('India','America','Ukraine','New Zealand','Australia','South Africa',
                 'Russia','Italy','Spain','Germany','Nepal','Ireland','Iran','Pakistan',
                 'Canada','India','America','Germany','China')
print(country_tuple[-5]) # 'Canada'
print(country_tuple[11]) # 'Ireland'
print(country_tuple[4:7]) # ('Aus','SA','Russia')
print(country_tuple[4:9:2]) # ('Aus',,'Russia','Spain')
print(country_tuple[::-5]) # ('China','Pakistan','Spain','New Zealand')
```

```
Canada
Ireland
('Australia', 'South Africa', 'Russia')
('Australia', 'Russia', 'Spain')
('China', 'Pakistan', 'Spain', 'New Zealand')
```

```python
# Finding the length of a tuple
country_tuple = ('India','America','Ukraine','New Zealand','Australia','South Africa',
                 'Russia','Italy','Spain','Germany','Nepal','Ireland','Iran','Pakistan',
                 'Canada','India','America','Germany','China')
print(len(country_tuple))
```

```
19
```

```python
# min(), max() , sum() -> Continuous Column [Numerical Columns]
num_tuple = (11,22,33,44,55,66,77,88,99,110,False,True) # 'mixed Dtype'
# int(False) = 0 & int(True) = 1
print(len(num_tuple)) # 12
print(min(num_tuple)) # False - 0
print(max(num_tuple)) # 110
print(sum(num_tuple)) # 606
```

```
12
False
110
606
```

```python
# Immutability of a Tuple
# We can't change the elements present on it.
# Tuples can't do : .append() , .extend() , .insert() , del , .clear() , .pop()
# There are some indirect method() that allows the tuple to manipulate the elements....
# Tuple -> Modifying to list()
country_tuple = ('India','America','Ukraine','New Zealand','Australia','South Africa',
                 'Russia','Italy','Spain','Germany','Nepal','Ireland','Iran','Pakistan',
                 'Canada','India','America','Germany','China')
conv_tuple_to_list = list(country_tuple) # 'class <list>'
print(type(conv_tuple_to_list))
```

```
<class 'list'>
```

```python
conv_tuple_to_list.append('Singapore')
print(conv_tuple_to_list)
```

```
['India', 'America', 'Ukraine', 'New Zealand', 'Australia', 'South Africa', 'Russia', 'Italy', 'Spain', 'Germa
ny', 'Nepal', 'Ireland', 'Iran', 'Pakistan', 'Canada', 'India', 'America', 'Germany', 'China', 'Singapore']
```

```python
country_tuple = tuple(conv_tuple_to_list)
print(country_tuple)
print(type(country_tuple)) # 'class <tuple>' # Protected Mode
```

```
('India', 'America', 'Ukraine', 'New Zealand', 'Australia', 'South Africa', 'Russia', 'Italy', 'Spain', 'Germa
ny', 'Nepal', 'Ireland', 'Iran', 'Pakistan', 'Canada', 'India', 'America', 'Germany', 'China', 'Singapore')
<class 'tuple'>
```

```python
# Concatenating a Tuple ('+' operator)
weekday_tuple = ('Mon','Tues','Wed','Thurs','Fri')
weekend_tuple = ('Sat','Sun')
week_tuple = weekday_tuple + weekend_tuple
print(week_tuple)
```

```
('Mon', 'Tues', 'Wed', 'Thurs', 'Fri', 'Sat', 'Sun')
```

```python
weekday_tuple = (['Mon','Tues'],)
weekend_tuple = ('Sat','Sun')
week_tuple = weekday_tuple + weekend_tuple
print(week_tuple)
```

```
(['Mon', 'Tues'], 'Sat', 'Sun')
```

```python
week_tuple[0] # 'list'
```

```
['Mon', 'Tues']
```

```python
type(week_tuple[0])
```

```
list
```

```python
week_tuple[0].extend(['Wed', 'Thurs', 'Fri'])
print(week_tuple)
```

```
(['Mon', 'Tues', 'Wed', 'Thurs', 'Fri'], 'Sat', 'Sun')
```

```python
weekday_tuple = ('Mon','Tues','Wed','Thurs','Fri')
weekend_tuple = ('Sat','Sun')
week_tuple = weekday_tuple[-3:] + weekend_tuple
print(week_tuple)
```

```
('Wed', 'Thurs', 'Fri', 'Sat', 'Sun')
```

```python
week_tuple[2] = "Friday" # TypeError: 'tuple' object does not support item assignment
```

```python
# Deleting an element in Tuple is not allowed.
# But we can delete the entire tuple....
weekday_tuple = ('Mon','Tues','Wed','Thurs','Fri')
print(weekday_tuple)
print(type(weekday_tuple))
```

```
('Mon', 'Tues', 'Wed', 'Thurs', 'Fri')
<class 'tuple'>
```

```python
del weekday_tuple[0] # Element
del weekday_tuple[-3:] # Sub-Tuple
TypeError: 'tuple' object doesn't support item deletion
```

```python
# Deleting the entire tuple
del weekday_tuple
```

```python
print(weekday_tuple)
NameError: name 'weekday_tuple' is not defined
```

```python
_tuple = 10,20,30,40,50,60,70,80,90
print(_tuple)
print(type(_tuple))
```

```
(10, 20, 30, 40, 50, 60, 70, 80, 90)
<class 'tuple'>
```

```python
# Unpacking a tuple
a,b,c = (10,20,30)
print(a) # 10
print(b) # 20
print(c) # 30
```

```
10
20
30
```

```python
# Unpacking a tuple
a,b,*c = (10,20,30,40,50,60,70,80,90)
print(a) # 10
print(b) # 20
print(c) # [30,40,50,60,70,80,90]
```

```
10
20
[30, 40, 50, 60, 70, 80, 90]
```

```
# Unpacking a tuple
a,*b,*c = (10,20,30,40,50,60,70,80,90)
print(a) # 10
print(b) # 20
print(c) # [30,40,50,60,70,80,90]
# SyntaxError: multiple starred expressions in assignment
```

```
# Unpacking a tuple
*a,b,c = 10,20,30,40,50,60,70,80,90
print(a) # [10,20,30,40,50,60,70]
print(tuple(a)) # (10,20,30,40,50,60,70)
print(b) # 80
print(c) # 90
```

```
[10, 20, 30, 40, 50, 60, 70]
(10, 20, 30, 40, 50, 60, 70)
80
90
```

```
# Unpacking a tuple
a,b,c = 10,20,30,40,50,60,70,80,90
print(a)
print(b)
print(c)
# ValueError: too many values to unpack (expected 3)
```

```
# Repeating Tuple (*)
_tuple = ('a','b','c','d','e')
print(_tuple * 3)
```

```
('a', 'b', 'c', 'd', 'e', 'a', 'b', 'c', 'd', 'e', 'a', 'b', 'c', 'd', 'e')
```

```
_tuple = 'a','b','c','d','e'
print(_tuple)
print(type(_tuple))
```

```
('a', 'b', 'c', 'd', 'e')
<class 'tuple'>
```

```
_tuple = 'a','b','c','d','e'
print(*_tuple)
print(type(_tuple))
```

```
a b c d e
<class 'tuple'>
```

```
_tuple[0]
```

```
'a'
```

```
type(_tuple[0])
```

```
str
```

```python
# Combining a Tuple (+)
_tuple1 = (1,2,3,4,5) # same d-type
_tuple2 = ("Coding",'Python','K',21.99,False,True) # mixed d-type
new_tuple = _tuple1 + _tuple2
print(new_tuple)
```

```
(1, 2, 3, 4, 5, 'Coding', 'Python', 'K', 21.99, False, True)
```

```python
# .count()
country_tuple = ('India','America','Ukraine','New Zealand','Australia','South Africa',
                 'Russia','Italy','Spain','Germany','Nepal','Ireland','Iran','Pakistan',
                 'Canada','India','America','Germany','China')
country_tuple.count('India')
```

```
2
```

```python
# .index()
country_tuple.index('Germany')
```

```
9
```

```python
# .index() -> to find the 2nd occurence , pass the 2nd argument as 1st Occ + 1
country_tuple.index('Germany',10)
```

```
17
```

```python
# Find the element in index that doesn't exist -> 'Error'
country_tuple.index('Norway')
# ValueError: tuple.index(x): x not in tuple
```